

Arrays – introduction, importing, creating, accessing, modifying & operations

एरेज़ (Arrays) – एरे का परिचय, मॉड्यूल इंपोर्ट करना, निर्माण, एक्सेस/मॉडिफाई और प्रमुख ऑपरेशन्स

एरे (Array) समान डेटा टाइप (same data type) के तत्वों का एक संग्रह है जो लगातार मेमोरी लोकेशन्स (contiguous memory locations) में स्टोर होते हैं। Python में एरे के साथ काम करने के लिए array मॉड्यूल प्रदान किया जाता है।

1. Introduction to Arrays | परिचय

- एरे समान डेटा टाइप के कई आइटम्स स्टोर करते हैं।
- एलिमेंट्स सन्निहित (contiguous) मेमोरी में रहते हैं।
- न्यूमेरिक डेटा के लिए लिस्ट की तुलना में अधिक कुशल और तेज़ हैं।

2. Importing array module | इंपोर्ट

```
import array as arr # import array module
```

3. Creating Arrays | एरे बनाना

array(typecode, [initializer]) द्वारा निर्माण:

```
import array as arr
a = arr.array('i') # empty integer array
b = arr.array('i', [10, 20, 30, 40]) # integer array
c = arr.array('f', [1.5, 2.5, 3.5]) # float array
d = arr.array('u', 'hello') # unicode array
```

Typecodes: 'i' → signed int, 'f' → float, 'u' → unicode

4. Accessing Elements | एक्सेस करना

```
b = arr.array('i', [10, 20, 30, 40])
print(b[0]) # 10 (first element)
print(b[2]) # 30
print(b[-1]) # 40 (last element)
```

5. Modifying Elements | बदलना

```
b = arr.array('i', [10, 20, 30, 40])
b[1] = 99 # modify element at index 1
b[3] = -10 # modify element at index 3
print(b) # array('i', [10, 99, 30, -10])
```

6. Basic Operations | मूल ऑपरेशन्स

```
a = arr.array('i', [10, 20, 30])
b = arr.array('i', [40, 50])
print(len(a)) # length of array (3)
print(a.buffer_info()) # (address, length in elements)
print(a.typecode) # type code ('i')
print(a.tolist()) # convert array to list
a.append(40) # add element at the end
a.extend(b) # add elements from another array
a.insert(1, 15) # insert 15 at index 1
a.remove(20) # remove first occurrence of 20
a.pop() # remove & return last element
a.reverse() # reverse array in-place
a.count(30) # count occurrences of 30
```

7. Arithmetic (Element-wise) | गणितीय ऑपरेशन्स

```
x = arr.array('i', [1, 2, 3])
y = arr.array('i', [10, 20, 30])
z = arr.array('i')
for i in range(len(x)):
    z.append(x[i] + y[i]) # element-wise addition
print(z) # array('i', [11, 22, 33])
```

8. Other Operations | अन्य ऑपरेशन्स

```
a = arr.array('i', [5, 1, 3, 2])
s = sorted(a) # sorted → [1, 2, 3, 5]
idx = a.index(3) # return index of first 3
cnt = a.count(2) # count occurrences of 2
del a[:] # clear all elements
print(a) # array('i') empty array
```

Key Points मुख्य बिंदु

- एरे केवल समान डेटा प्रकार (Homogeneous elements) को संग्रहीत करते हैं।
- एरे निर्माण के लिए arr.array(typecode, initializer) कंस्ट्रक्टर का उपयोग करें।
- तत्वों को 0-आधारित इंडेक्स (0-based indexing) द्वारा एक्सेस व संशोधित किया जाता है।
- विभिन्न मेथड्स जैसे append(), insert(), remove(), pop(), extend(), reverse() समर्थित हैं।
- संख्यात्मक गणनाओं के लिए एरे मेमोरी-कुशल और लिस्ट की तुलना में अधिक तीव्र होते हैं।

© Vinay Soni

Arrays – Introduction to Arrays

एरेज़ (Arrays) – एरे का विस्तृत परिचय, आंतरिक संरचना एवं कार्यप्रणाली

एरे (Array) समान डेटा टाइप के तत्वों का एक व्यवस्थित संग्रह है जो कंप्यूटर की सन्निहित मेमोरी लोकेशन्स (contiguous memory locations) में स्टोर होता है। Python में एरे के उपयोग के लिए array मॉड्यूल का प्रयोग किया जाता है।

1. Introduction to Arrays | एरे की प्रमुख विशेषताएँ

- समान डेटा प्रकार (Homogeneous Elements):** एरे में सभी तत्व एक ही प्रकार और समान मेमोरी आकार के होते हैं (जैसे केवल इंटीजर्स या केवल फ्लोट्स)।
- सन्निहित मेमोरी (Contiguous Memory):** सभी तत्व मेमोरी में एक के बाद एक लगातार ब्लॉक्स में स्टोर होते हैं, जिससे इंडेक्स द्वारा एक्सेस बहुत तेज़ $O(1)$ होता है।
- मेमोरी एवं परफॉर्मेंस दक्षता:** बड़े संख्यात्मक डेटा (numeric data) के लिए एरे सामान्य Python लिस्ट की तुलना में कम मेमोरी लेते हैं और तेज़ निष्पादन देते हैं।
- स्ट्रिक्ट टाइप बाइंडिंग:** निर्माण के समय निर्दिष्ट टाइपकोड के अतिरिक्त किसी अन्य प्रकार का मान एरे में स्टोर नहीं किया जा सकता।
- डायनामिक रिसाइज़िंग:** Python एरे C-एरेज़ के विपरीत आवश्यकतानुसार आकार में बढ़ (grow) या घट (shrink) सकते हैं।

Visual Memory Representation | एरे की मेमोरी संरचना एवं इंडेक्सिंग

मान लीजिए 4-बाइट पूर्णांकों (signed int 'i') का एरे `arr = array('i', [10, 20, 30, 40, 50])` है:



मेमोरी पता गणना सूत्र (Address Calculation):

$Address(arr[i]) = Base_Address + (i \times Item_Size)$

प्रत्येक तत्व का साइज़ समान होने के कारण CPU तुरंत किसी भी इंडेक्स का पता गणना कर सीधे एक्सेस कर लेता है ($O(1)$ Time)।

```
import array as arr
a = arr.array('i', [10, 20, 30, 40, 50])
print("Item size (bytes):", a.itemsize)      # 4
print("Buffer info:", a.buffer_info())      # (address, 5)
```

Python List vs Array Comparison | लिस्ट और एरे में मुख्य अंतर

फीचर (Feature)	Python List (लिस्ट)	Python Array (एरे)
डेटा प्रकार (Data Type)	हेटरोजेनियस (Heterogeneous) - विभिन्न प्रकार के डेटा	होमोजेनियस (Homogeneous) - केवल एक समान प्रकार
मॉड्यूल आवश्यकता	बिल्ट-इन (Built-in) - सीधे उपलब्ध	<code>import array</code> मॉड्यूल की आवश्यकता होती है
मेमोरी प्रबंधन	पॉइंटर्स की लिस्ट (Overhead अधिक होता है)	रॉ बाइनरी बफर (Compact एवं Memory Efficient)
संख्यात्मक परफॉर्मेंस	सामान्य (धीमी गति)	अत्यधिक तीव्र (C-स्पीड के निकट)
उपयोग क्षेत्र	सामान्य प्रोग्रामिंग एवं मिश्रित डेटा संग्रह	न्यूमेरिकल प्रोसेसिंग, हार्डवेयर/फाइल बफर

Key Points मुख्य बिंदु

- एरे केवल समान प्रकार (same type) के तत्वों को संग्रहित करता है।
- इंडेक्स 0 से शुरू होकर $N - 1$ तक जाता है; ऋणात्मक इंडेक्स -1 से पीछे की ओर चलता है।
- सन्निहित मेमोरी लोकेशन के कारण एरे तत्वों का एक्सेस समय अत्यंत तीव्र होता है।

© Vinay Soni

Arrays – Importing Array Module

एरेज़ (Arrays) – array मॉड्यूल को इंपोर्ट करना एवं उपयोग विधियाँ

Python में एरे लिस्ट (lists) की तरह इन-बिल्ट नहीं होते हैं। एरे के साथ काम करने के लिए हमें सबसे पहले array मॉड्यूल को इंपोर्ट करना होता है। यह मॉड्यूल हमें array ऑब्जेक्ट और C-लेवल डेटा बफ़र्स प्रदान करता है।

1. Importing array module | मॉड्यूल इंपोर्ट करने की 4 मुख्य विधियाँ

Python में import स्टेटमेंट द्वारा एरे मॉड्यूल को निम्न प्रकार से इंपोर्ट किया जा सकता है:

इंपोर्ट सिंटैक्स (Syntax)	ऑब्जेक्ट निर्माण का तरीका (Usage)	विवरण एवं प्राथमिकता (Description)
<code>import array as arr</code>	<code>a = arr.array('i', [1, 2, 3])</code>	सर्वोत्तम तरीका (Recommended): छोटा, स्पष्ट उपनाम (alias) जो नेमस्पेस को सुरक्षित रखता है।
<code>import array</code>	<code>a = array.array('i', [1, 2, 3])</code>	मानक तरीका: प्रत्येक बार मॉड्यूल का पूरा नाम लिखना पड़ता है।
<code>from array import array</code>	<code>a = array('i', [1, 2, 3])</code>	प्रत्यक्ष आयात: केवल <code>array</code> क्लास को सीधे वर्तमान नेमस्पेस में इंपोर्ट करता है।
<code>from array import *</code>	<code>a = array('i', [1, 2, 3])</code>	सभी घटकों को इंपोर्ट करता है (बड़े प्रोजेक्ट्स में नेम टकराव के कारण अनुशंसित नहीं)।

Example: Practical Usage & Inspection | व्यावहारिक उदाहरण एवं परीक्षण

```
# 1. Alias के साथ इंपोर्ट करें (मानक परंपरा)
import array as arr

# 2. arr.array() का उपयोग करके इंटीजर एरे बनाएँ
numbers = arr.array('i', [10, 20, 30, 40])

# 3. एरे, उसका डेटा टाइप और टाइपकोड प्रिंट करें
print("Array Object :", numbers)
print("Type Class   :", type(numbers))
print("Typecode    :", numbers.typecode)
print("Item size   :", numbers.itemsize, "bytes")
```

OUTPUT | आउटपुट

```
Array Object :
array('i', [10, 20, 30, 40])

Type Class :
<class 'array.array'>

Typecode : 'i'
Item size : 4 bytes
```

Why use Array Module? | array मॉड्यूल का उपयोग क्यों करें?

स्मृति दक्षता (Memory Efficiency):

Python लिस्ट्स प्रत्येक आइटम के लिए 8-बाइट पॉइंटर और ऑब्जेक्ट हेडर रखती हैं। एरे मॉड्यूल डायरेक्ट C-टाइप बाइनरी स्टोर करता है, जिससे 60% से 75% मेमोरी बचती है।

डेटा सुरक्षा व अखंडता (Type Safety):

समान टाइपकोड सुनिश्चित करता है कि गलती से भी कोई अवांछित डेटा टाइप (जैसे स्ट्रिंग या ऑब्जेक्ट) संख्यात्मक एरे में न डाला जा सके, जिससे रनटाइम बग्स रुकते हैं।

Key Points मुख्य बिंदु

- एरे ऑब्जेक्ट बनाने से पहले `import array as arr` करना अनिवार्य है।
- `array.array` क्लास एक सी-कम्पैटिबल कंटीन्यूअस बफर उपलब्ध कराती है।
- यह सामान्य लिस्ट से भिन्न है क्योंकि इसमें केवल एक समान डेटा प्रकार ही स्टोर हो सकता है।

Best Practice / सर्वोत्तम नियम : Python कोड में हमेशा `import array as arr` का प्रयोग करें ताकि कोड संक्षिप्त, पठनीय और मानक बना रहे।

© Vinay Soni

Arrays – Creating Arrays

एरेज़ (Arrays) – array() कंस्ट्रक्टर और टाइपकोड्स द्वारा एरे निर्माण

array मॉड्यूल इंपोर्ट करने के बाद, हम array() कंस्ट्रक्टर का उपयोग करके नए एरे का निर्माण करते हैं। निर्माण के समय डेटा का **Typecode** और प्रारंभिक मान (initializer) देना होता है।

1. Creating Arrays | निर्माण सिंटैक्स एवं घटक

```
arr = array(typecode, [initializer]) # typecode is one of 'i', 'f', 'u', 'd', 'b' etc.
```

जहाँ (Where):

- typecode → एरे में स्टोर किए जाने वाले तत्वों का प्रकार और मेमोरी साइज़ निर्धारित करने वाला एकल वर्ण (Single character)।
- initializer → एरे को इनिशियलाइज़ करने के लिए वैकल्पिक लिस्ट, टपल, स्ट्रिंग या अन्य इटरेबल।

Standard Typecodes Guide | मानक टाइपकोड्स संदर्भ तालिका

Typecode	C Type	Python Data Type	न्यूनतम साइज़	स्वीकृत सीमा / विवरण (Value Range)
'b' / 'B'	signed / unsigned char	int (पूर्णांक)	1 Byte	-128 से 127 / 0 से 255 (बाइट्स डेटा)
'u'	Py_UNICODE	str (यूनिकोड)	2 / 4 Bytes	यूनिकोड कैरेक्टर स्ट्रिंग जैसे 'hello'
'h' / 'H'	signed / unsigned short	int (शॉर्ट इंट)	2 Bytes	-32,768 से 32,767 / 0 से 65,535
'i' / 'I'	signed / unsigned int	int (मानक पूर्णांक)	2 / 4 Bytes	-2,147,483,648 से 2,147,483,647
'l' / 'L'	signed / unsigned long	int (लॉन्ग इंट)	4 / 8 Bytes	बड़ी पूर्णांक संख्याएँ
'f'	float	float (दशमलव)	4 Bytes	सिंगल प्रिसिजन दशमलव (e.g. 1.5, 2.5)
'd'	double	float (डबल फ्लोट)	8 Bytes	डबल प्रिसिजन फ्लोट (उच्च सटीकता दशमलव)

Examples of Array Creation | एरे निर्माण के उदाहरण

```
import array as arr

# 1. खाली इंटीजर एरे बनाना
a = arr.array('i')

# 2. लिस्ट इनिशियलाइज़र के साथ इंटीजर एरे
b = arr.array('i', [10, 20, 30, 40])

# 3. फ्लोटिंग पॉइंट एरे
c = arr.array('f', [1.5, 2.5, 3.5])

# 4. यूनिकोड कैरेक्टर एरे
d = arr.array('u', 'hello')
```

OUTPUT / RESULT | आउटपुट

```
a → array('i')
b → array('i', [10, 20, 30, 40])
c → array('f', [1.5, 2.5, 3.5])
d → array('u', 'hello')
```

TypeError से सावधान : यदि हम किसी टाइपकोड में असंगत प्रकार का मान डालने का प्रयास करते हैं (जैसे arr.array('i', [1.5, 2.5])), तो Python TypeError: integer argument expected, got float उत्पन्न करता है।

Key Points मुख्य बिंदु

- डेटा प्रकार के अनुसार उचित typecode निर्दिष्ट करना अनिवार्य है।
- खाली एरे बनाने के लिए बिना इनिशियलाइज़र के केवल टाइपकोड दें: arr.array('i')।
- इनिशियलाइज़र के रूप में लिस्ट, टपल, या स्ट्रिंग का उपयोग किया जा सकता है।

© Vinay Soni

Arrays – Accessing & Modifying Elements

एरेज़ (Arrays) – इंडेक्सिंग एवं स्लाइसिंग द्वारा तत्वों को एक्सेस और संशोधित करना

एरे में प्रत्येक तत्व का एक निश्चित इंडेक्स होता है। हम **0-आधारित इंडेक्सिंग (0-based indexing)** का उपयोग करके तत्वों को एक्सेस (पढ़) और मॉडिफाई (परिवर्तित) कर सकते हैं।

1. Accessing Elements | तत्व एक्सेस करना

स्क्वायर ब्रैकेट `arr[index]` का उपयोग करके मान प्राप्त करें:

```
import array as arr
a = arr.array('i', [10, 20, 30, 40, 50])

print(a[0]) # Output: 10 (पहला तत्व)
print(a[2]) # Output: 30 (तीसरा तत्व)
print(a[4]) # Output: 50 (पॉचवाँ तत्व)
print(a[-1]) # Output: 50 (अंतिम तत्व)
```

चेतावनी (IndexError): यदि कोई ऐसा इंडेक्स एक्सेस किया जाए जो मौजूद नहीं है, तो Python `IndexError` फेंकता है।

त्रुटि उदाहरण (IndexError Example):

```
a = arr.array('i', [10, 20, 30, 40])
print(a[1]) # Output: 20 (वैध)
print(a[4]) # IndexError: array index out of range
# (मान्य इंडेक्स: 0 से 3)
```

2. Modifying Elements | तत्व बदलना

इंडेक्स पर नया मान असाइन करके तत्व बदला जा सकता है:

```
import array as arr
a = arr.array('i', [10, 20, 30, 40, 50])

a[2] = 300 # change element at index 2
a[4] = 500 # change element at index 4
print(a)
# Output: array('i', [10, 20, 300, 40, 500])
```

अतिरिक्त उदाहरण (Another Example):

```
a = arr.array('i', [5, 15, 25, 35])

a[1] = 150 # modify element at index 1
print(a) # array('i', [5, 150, 25, 35])

a[3] = 350 # modify element at index 3
print(a) # array('i', [5, 150, 25, 350])
```

RESULT | परिणाम

इंडेक्स 1 पर 15 बदलकर 150 हुआ।
इंडेक्स 3 पर 35 बदलकर 350 हुआ।

Array Slicing | एरे स्लाइसिंग (तत्वों का उप-समूह प्राप्त करना)

```
a = arr.array('i', [10, 20, 30, 40, 50, 60])
print(a[1:4]) # index 1 to 3 → array('i', [20, 30, 40])
print(a[:3]) # start to 2 → array('i', [10, 20, 30])
print(a[::2]) # step of 2 → array('i', [10, 30, 50])
print(a[::-1]) # reverse view → array('i', [60, 50, 40, 30, 20, 10])
```

SLICING SYNTAX

`arr[start : stop : step]`

स्लाइसिंग से मूल एरे के एक हिस्से का नया एरे ऑब्जेक्ट प्राप्त होता है।

Indexing Rules Summary | इंडेक्सिंग के प्रमुख नियम

इंडेक्स प्रकार	दिशा (Direction)	दायरा (Range)	उदाहरण
धनात्मक (Positive)	बाएँ से दाएँ (शुरुआत से अंत)	0 से <code>len(a) - 1</code>	<code>a[0]</code> पहला तत्व, <code>a[1]</code> दूसरा तत्व
ऋणात्मक (Negative)	दाएँ से बाएँ (अंत से शुरुआत)	-1 से <code>-len(a)</code>	<code>a[-1]</code> अंतिम तत्व, <code>a[-2]</code> सेकंड लास्ट

Key Points मुख्य बिंदु

- तत्व एक्सेस करने के लिए `a[index]` का उपयोग किया जाता है।
- तत्व बदलने के लिए `a[index] = new_value` का उपयोग किया जाता है।
- नया मान उसी डेटा टाइप का होना चाहिए जो एरे का टाइपकोड है।
- अमान्य इंडेक्स पर एक्सेस या मॉडिफाई करने से `IndexError` आता है।

© Vinay Soni

Arrays – Operations & Built-in Methods

एरेज़ (Arrays) – प्रमुख ऑपरेशन्स, इन-बिल्ट मेथड्स एवं संपूर्ण उदाहरण

Python एरे डेटा प्रोसेसिंग और मैनिपुलेशन के लिए विभिन्न बिल्ट-इन मेथड्स और ऑपरेशन्स का समर्थन करते हैं। नीचे 8 प्रमुख ऑपरेशन्स दिए गए हैं:

1. Traversing / Iterating | इटरेट करना

for लूप द्वारा एरे के सभी तत्वों को इटरेट करें:

```
import array as arr
a = arr.array('i', [10, 20, 30, 40])
for x in a:
    print(x, end=" ")
# Output: 10 20 30 40
```

2. Length of Array | एरे की लंबाई

len() द्वारा कुल तत्वों की संख्या प्राप्त करें:

```
import array as arr
a = arr.array('i', [10, 20, 30, 40])
print(len(a))
# Output: 4
```

3. Adding Elements | तत्व जोड़ना (append)

append() द्वारा अंत में एक तत्व जोड़ें:

```
import array as arr
a = arr.array('i', [10, 20, 30])
a.append(40)
print(a)
# Output: array('i', [10, 20, 30, 40])
```

4. Extending Array | विस्तार करना (extend)

extend() द्वारा इटरेबल से कई तत्व जोड़ें:

```
import array as arr
a = arr.array('i', [10, 20])
a.extend([30, 40, 50])
print(a)
# Output: array('i', [10, 20, 30, 40, 50])
```

5. Searching Elements | खोजना (index)

index() द्वारा मान का पहला इंडेक्स प्राप्त करें:

```
import array as arr
a = arr.array('i', [10, 20, 30, 40])
print(a.index(30)) # Output: 2
# a.index(50) raises ValueError
```

6. Counting Occurrences | गणना (count)

count() द्वारा तत्व की आवृत्ति ज्ञात करें:

```
import array as arr
a = arr.array('i', [10, 20, 20, 30, 20, 40])
print(a.count(20))
# Output: 3
```

7. Removing Elements | हटाना (remove)

remove() पहली उपस्थिति को हटाता है:

```
import array as arr
a = arr.array('i', [10, 20, 30, 20])
a.remove(20) # removes first 20
print(a)
# Output: array('i', [10, 30, 20])
```

8. Reversing Array | उलटाना (reverse)

reverse() क्रम को इन-प्लेस उलट देता है:

```
import array as arr
a = arr.array('i', [10, 20, 30, 40])
a.reverse()
print(a)
# Output: array('i', [40, 30, 20, 10])
```

Additional Essential Methods | अतिरिक्त महत्वपूर्ण मेथड्स

मेथड (Method)	सिंटैक्स (Syntax)	कार्य (Action)	संभावित एरर (Exception)
insert()	a.insert(index, val)	निर्दिष्ट इंडेक्स पर तत्व सम्मिलित करता है	TypeError (यदि प्रकार गलत हो)
pop()	a.pop([index])	इंडेक्स से तत्व हटाकर लौटाता है (डिफॉल्ट अंतिम)	IndexError (यदि खाली हो या अवैध इंडेक्स)
tolist()	lst = a.tolist()	एरे को मानक Python लिस्ट में परिवर्तित करता है	-
buffer_info()	addr, count = a.buffer_info()	मेमोरी एड्रेस और तत्वों की संख्या का टपल लौटाता है	-

Key Points मुख्य बिंदु

- Python एरे कई शक्तिशाली इन-बिल्ट मेथड्स का समर्थन करते हैं जो मेमोरी-कुशल होते हैं।
- remove() और index() का उपयोग करते समय यदि तत्व नहीं मिलता है तो ValueError उत्पन्न होता है।
- एरेज़ पर कुशल काम करने के लिए len(), append(), extend(), insert() और pop() सबसे अधिक उपयोग किए जाते हैं।

© Vinay Soni